

THE COWARD KING

Reflections

Tactical Roguelite Board Game



José Abel Domínguez Rish A01781852

Leonardo André Flores Mendoza A01787221

Nicolás Casillas Larrañaga A01787292

Game Title	The Coward King
Genre	Roguelite Tactical Turn-Based Board Game
Platform	PC (Desktop)
Target Audience	Strategy & roguelite fans, 12+
Team Size/Name	3 developers - Silent Crown
MVP Timeline	8 weeks
Document Status	Section 2: Game Design completed

Selected Strategy

We developed **The Coward King** as a turn-based tactical roguelite with cards. We chose this strategy because it combines resource management, positioning, randomness, and replayability while making the King's cowardice part of the gameplay.

Challenges and Solutions

- **Game balance:**
During testing, some hordes were too easy because the player could remain protected behind allies. We adjusted the AP limit, enemy quantities, attack order, obstacles, bosses, and added the Desperation system to force the King to move.
- **Enemy movement and collisions:**
Enemies could overlap or become blocked incorrectly. We improved tile occupation checks and movement rules so enemies, allies, obstacles, and the King interact correctly on the board.
- **Card system:**
Balancing 15 cards with different costs, effects, upgrades, and placement limits was challenging. We tested their behavior, adjusted statistics, added reusable cards, and implemented random hands with card retention between hordes.
- **Level progression:**
Creating three levels that felt different required more than changing the background. We introduced stronger enemies, additional spawn directions, more obstacles, higher enemy limits, and unique bosses.
- **Boss implementation:**
The bosses initially became trapped by obstacles or lacked enough pressure. We removed obstacles from boss encounters, made bosses occupy four tiles, added periodic movement, and allowed them to summon stronger enemies.
- **Database and API integration:**
Connecting the frontend, backend, API, and MySQL database required coordinating different data formats. We created endpoints for users, runs, upgrades, cards, statistics, and rankings, then connected them through `fetch` requests.
- **Visual and audio consistency:**
The interface originally mixed different visual styles. We standardized the game around medieval pixel art, redesigned the menu and cards, added sprites and animations, and implemented music and sound effects.
- **Team collaboration and version control:**
Working in different branches caused conflicts and occasionally resulted in lost changes. We recovered files, improved documentation, divided tasks more clearly, and used README and GDD updates to keep track of the project's current state.
- **Code organization:**
The original prototype placed most logic in one file, making it difficult to maintain. We separated responsibilities into modules for turns, combat, movement, entities, rendering, UI, audio, API communication, and board logic.
-

Main Lessons Learned

- Plan the architecture before adding complex mechanics.
- Test gameplay continuously.
- Keep code, database, and documentation synchronized.
- Communicate changes clearly within the team.
- Use version control carefully to avoid losing work.

Individual Conclusions

José Abel Domínguez Rish:

I learned how gameplay mechanics, user interface, visual design, audio, and documentation must work together to create a complete and coherent videogame. I also improved my ability to identify gameplay problems, balance mechanics, organize ideas, and transform feedback into practical improvements.

Leonardo André Flores Mendoza:

I learned the importance of connecting the frontend, backend, API, and database correctly. Working on user accounts, statistics, runs, and persistent upgrades helped me understand how data is stored and communicated across the different layers of a complete software project.

Nicolás Casillas Larrañaga:

I learned that modular code, continuous testing, and clear teamwork make a large project easier to maintain and expand. Dividing responsibilities and communicating changes helped us integrate different systems while preserving the game's mechanics, visual identity, and overall stability.

Final Conclusion

The project helped us transform a simple initial prototype into a more complete videogame with tactical mechanics, card-based strategy, level and horde progression, a consistent medieval pixel-art identity, animated sprites, music, sound effects, an API, and database integration.

Throughout development, we learned that creating a videogame requires much more than programming individual mechanics. It also involves continuous testing, gameplay balancing, visual design, documentation, data management, version control, and clear communication between team members.

Our greatest lesson was that successful development depends equally on technical knowledge, iteration, organization, creativity, and teamwork. Although we faced several challenges, each problem helped us improve both the project and our ability to collaborate. The final result represents the growth of the game and the knowledge we gained during the entire development process.